SMART OUTCOME PREDICTOR

Ensemble Learning Project Documentation

Bagging • Boosting • Voting • Stacking

Table of Contents

1. Project Overview

2. Dataset Description

3. Part A: Conceptual Foundation (Tasks 1–5)

4. Part B: Dataset Understanding & Preparation (Tasks 6–8)

5. Part C: Bagging (Tasks 9–11)

6. Part D: Boosting Algorithms (Tasks 12–17)

7. Part E: Voting & Stacking Ensembles (Tasks 18–21)

8. Part F: Model Evaluation & Comparison (Tasks 22, 29, 30)

9. Part G: Final Analysis & Reporting (Tasks 31–32)

10. Deployment and Submission

11. Conclusion

1. Project Overview

1.1 Objective

The objective of this project is to design a robust ensemble-based machine learning system that predicts outcomes accurately by combining multiple weak and strong learners. The project explores bagging, boosting, voting and stacking, and shows how ensembles improve generalization, stability and performance in both classification and regression tasks.

1.2 Problem Statement

As a Machine Learning Engineer at an EdTech analytics company, the task is to build a Smart Outcome Predictor that can:

- **Classification:** predict whether a student will successfully complete an online course (completion_status).
- **Regression:** predict the final performance score of a student (final_score, 0–100).

Single models produce unstable results, so an ensemble-based solution is designed to improve accuracy and robustness by combining multiple models.

1.3 Tools and Technologies

Purpose	Tool / Library
Language / IDE	Python 3, Jupyter Notebook
Data handling	pandas, NumPy, openpyxl (read .xlsx)
Visualization	Matplotlib, Seaborn
Machine learning	scikit-learn (Bagging, AdaBoost, Gradient Boosting, Voting, Stacking, Pipelines)
Boosting library	XGBoost
Deployment	Streamlit, joblib (saved pipelines), GitHub

1.4 Project Workflow

1. Load dataset and perform EDA (target distribution, missing values, correlation).
2. Define features and targets; split data 80/20 with a fixed random seed.
3. Build a preprocessing Pipeline (imputation, scaling, encoding).
4. Train base models, then Bagging, AdaBoost, XGBoost, Voting and Stacking models.
5. Evaluate with classification and regression metrics; compare all ensembles.
6. Select the best model per task, save the pipelines and deploy through a Streamlit app.

2. Dataset Description

The dataset (Smart_Outcome_Predictor_Dataset_5200.xlsx) contains 5,200 student records and 19 columns: demographic indicators, course details, engagement metrics, assessment scores and the two target variables.

Column	Type	Description
student_id	Integer	Unique student identifier (dropped, not a feature)
age	Integer	Student age
country_region	Categorical	Africa, Americas, Asia, Europe, Oceania
device_type	Categorical	Laptop, Mobile, Tablet
education_background	Categorical	HighSchool, Undergrad, Graduate, WorkingPro
course_level	Categorical	Beginner, Intermediate, Advanced
course_category	Categorical	Business, Data, Design, Marketing, Programming
course_start_date	Date	Converted to start_month
week_of_year	Integer	Week in which the course started
sessions	Integer	Number of learning sessions
time_spent_hours	Float	Total hours spent (112 missing)
videos_watched	Integer	Number of videos watched
quiz_attempts	Integer	Number of quiz attempts
assignments_submitted	Integer	Assignments submitted
forum_posts	Integer	Discussion forum posts
avg_quiz_score	Float	Average quiz score (81 missing)
attendance_rate	Float	Attendance, 0 to 1 (80 missing)
completion_status	Target (clf)	0 = Not Completed, 1 = Completed
final_score	Target (reg)	Final performance score, 0–100

Assumptions made

- student_id is only an identifier, so it is dropped.
- course_start_date is converted into the numeric feature start_month.
- Each target is not used as a feature for the other task, which avoids data leakage.
- Missing values are imputed with the median inside the pipeline (fitted on training data only).
- The same random seed (42) and the same train/test rows are used for both tasks, so the results are comparable.

3. Part A: Conceptual Foundation (Tasks 1–5)

Q1. What are Ensemble Learning Methods and why are they effective?

Ensemble learning combines the predictions of multiple models (base learners) to build one stronger model. They are effective because different models make different errors; when the predictions are averaged, voted or chained, the errors cancel out. The result is higher accuracy, better generalization and more stable predictions than a single model.

Q2. Difference between Bagging, Boosting and Voting

Method	How it works	Reduces	Example
Bagging	Many models trained in parallel on random bootstrap samples; outputs averaged or voted	Variance	Random Forest
Boosting	Models trained sequentially; each new model focuses on the mistakes of the previous ones	Bias	AdaBoost, XGBoost
Voting	Different model types trained on the same data; combined by majority vote or average probability	Individual model weaknesses	LR + KNN + Tree + RF

Q3. What is the Bias–Variance Trade-off and how do ensembles address it?

Bias is the error from a model that is too simple (underfitting). Variance is the error from a model that is too sensitive to the training data (overfitting). Reducing one usually increases the other. Bagging lowers variance by averaging many deep trees, and boosting lowers bias by adding learners that fix the remaining errors, so ensembles reach a better balance.

Q4. Explain Voting Classifier and Stacking Ensemble

- **Voting Classifier:** combines diverse base classifiers. Hard voting takes the majority class; soft voting averages the predicted probabilities.
- **Stacking:** the out-of-fold predictions of the base models become input features for a meta-learner (for example Logistic Regression), which learns how best to combine them.

Q5. Compare AdaBoost and XGBoost conceptually

Aspect	AdaBoost	XGBoost
Idea	Increases weights of misclassified samples each round	Fits each tree to the gradient (residual error) of the loss
Weak learner	Usually decision stumps	Deeper regularized trees
Regularization	None built in; sensitive to noise	L1/L2 penalties, shrinkage, subsampling
Missing values	Needs imputation	Handled natively
Speed	Sequential, slower	Optimized, parallel tree building
Final output	Weighted vote of learners	Sum of tree outputs

4. Part B: Dataset Understanding & Preparation (Tasks 6–8)

4.1 Exploratory Data Analysis

The dataset has 5,200 rows and 19 columns. Three numeric columns contain missing values (time_spent_hours: 112, avg_quiz_score: 81, attendance_rate: 80), which is about 1.5–2% of the data. There are no duplicate rows.

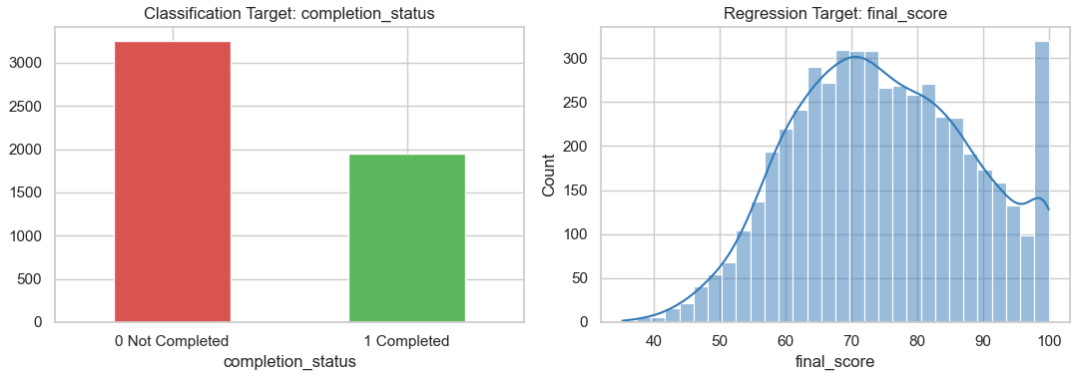


Figure 1: Distribution of the two target variables

Interpretation: The classes are moderately imbalanced (62.5% Not Completed vs 37.5% Completed), so a stratified split is used and Precision, Recall and F1 are reported along with Accuracy. final_score is roughly bell-shaped between 0 and 100.



Figure 2: Correlation heatmap of numeric features

Interpretation: The heatmap shows which engagement and assessment features move together with completion_status and final_score.

4.2 Task 6: Identify features and target variables

Task	Features (X)	Target (y)
Classification	11 numeric + 5 categorical	completion_status
Regression	Same 16 features	final_score

- **Numeric (11):** age, week_of_year, start_month, sessions, time_spent_hours, videos_watched, quiz_attempts, assignments_submitted, forum_posts, avg_quiz_score, attendance_rate
- **Categorical (5):** country_region, device_type, education_background, course_level, course_category
- **Dropped:** student_id (identifier), course_start_date (converted to start_month), and the other target (leakage)

4.3 Task 7: Train–test split

```
X_train, X_test, yc_train, yc_test, yr_train, yr_test = train_test_split(
    X, y_clf, y_reg, test_size=0.20, random_state=42, stratify=y_clf)
```

Set	Rows	Features	Completed ratio
Train	4,160	16	0.375
Test	1,040	16	0.375

One 80/20 split, stratified on completion_status, is shared by both tasks, so classification and regression are evaluated on the same students.

4.4 Task 8: Preprocessing

```
num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')),
                     ('scaler',  StandardScaler())])
cat_pipe = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
                     ('onehot',  OneHotEncoder(handle_unknown='ignore'))])
preprocessor = ColumnTransformer([('num', num_pipe, NUM_FEATURES),
                                  ('cat', cat_pipe, CAT_FEATURES)])
```

After preprocessing the training data has 31 columns (11 scaled numeric + 20 one-hot encoded). Because everything is wrapped in a Pipeline, preprocessing is fitted on the training data only (no leakage), and the same preprocessing is saved together with the model for the Streamlit app.

5. Part C: Bagging (Tasks 9–11)

5.1 Task 9: Bagging Classifier

```
BaggingClassifier(estimator=DecisionTreeClassifier(random_state=42),
                  n_estimators=100, max_samples=0.8, bootstrap=True,
                  n_jobs=-1, random_state=42)
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree (single)	0.6250	0.5000	0.5179	0.5088	0.6036
Bagging Classifier	**0.7163**	**0.6401**	**0.5564**	**0.5953**	**0.7727**

5.2 Task 10: Bagging Regressor

Model	MAE	RMSE	R^2
Decision Tree (single)	11.1198	14.1222	-0.0955
Bagging Regressor	**8.2389**	**10.2893**	**0.4184**

5.3 Task 11: Bagging vs single base model

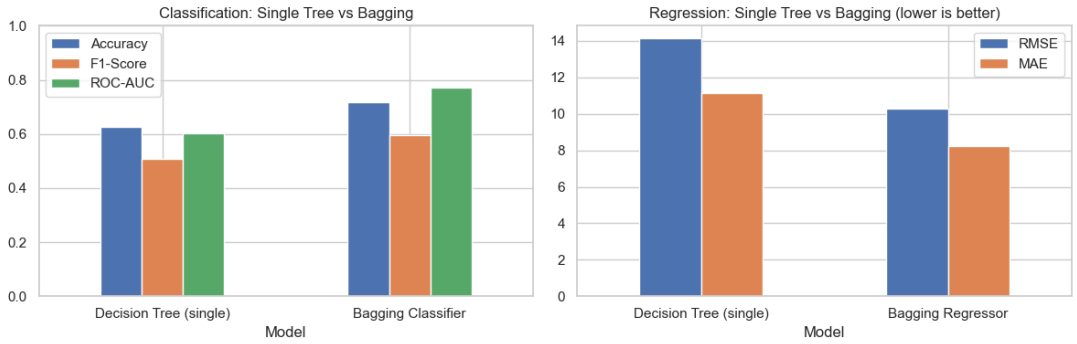


Figure 3: Single Decision Tree vs Bagging (classification and regression)

5-fold cross-validation ROC-AUC: Single Tree 0.6143 ± 0.0097, Bagging 0.7593 ± 0.0103.

> **Interpretation:** A single fully-grown tree has high variance (it memorizes the training data). Bagging averages 100 trees trained on different bootstrap samples, which reduces variance. ROC-AUC rises from 0.604 to 0.773, and regression RMSE falls from 14.12 to 10.29.

6. Part D: Boosting Algorithms (Tasks 12–17)

6.1 AdaBoost

```
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1),
                   n_estimators=200, learning_rate=0.5, random_state=42)
AdaBoostRegressor(estimator=DecisionTreeRegressor(max_depth=3),
                  n_estimators=200, learning_rate=0.5, random_state=42)
```

Task 12: AdaBoost Classifier | Task 13: AdaBoost Regressor

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Stump (weak learner)	0.6625	0.5572	0.4872	0.5198	0.6274
AdaBoost Classifier	**0.7298**	**0.6730**	**0.5436**	**0.6014**	**0.7870**

Model	MAE	RMSE	R^2
Shallow Tree depth=3 (weak learner)	9.3800	11.5976	0.2611
AdaBoost Regressor	**8.6941**	**10.6530**	**0.3766**

6.2 Task 14: How weak learners improve sequentially

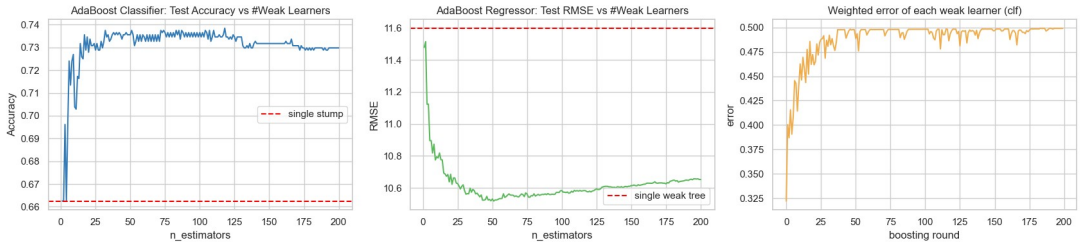


Figure 4: AdaBoost performance versus number of weak learners

Weak learners	1	10	50	200
Classifier accuracy	0.662	0.704	0.736	0.730
Regressor RMSE	11.48	10.79	10.53	10.65

Interpretation: In each round AdaBoost increases the weight of samples that earlier learners got wrong, so the next weak learner focuses on the hard students. Performance improves quickly in the first rounds (a single stump reaches 0.662 accuracy, the ensemble about 0.73) and then flattens. The weighted error of each new learner rises towards 0.5 because later learners deal with harder, re-weighted samples.

6.3 XGBoost (Tasks 15 and 16)

```
XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=4, subsample=0.8,
              colsample_bytree=0.8, reg_lambda=1.0, eval_metric='logloss',
random_state=42)
XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=4, subsample=0.8,
             colsample_bytree=0.8, reg_lambda=1.0, random_state=42)
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost Classifier	0.7317	0.6628	0.5795	0.6183	0.7812
Gradient Boosting Classifier	0.7356	0.6706	0.5795	0.6217	0.7904

Model	MAE	RMSE	R^2
XGBoost Regressor	8.2417	10.2149	0.4268
Gradient Boosting Regressor	8.1631	10.1468	0.4344

6.4 Task 17: Performance and robustness of boosting models

Robustness is measured with 5-fold cross-validation on the training data: the mean shows performance and the standard deviation shows stability across folds.

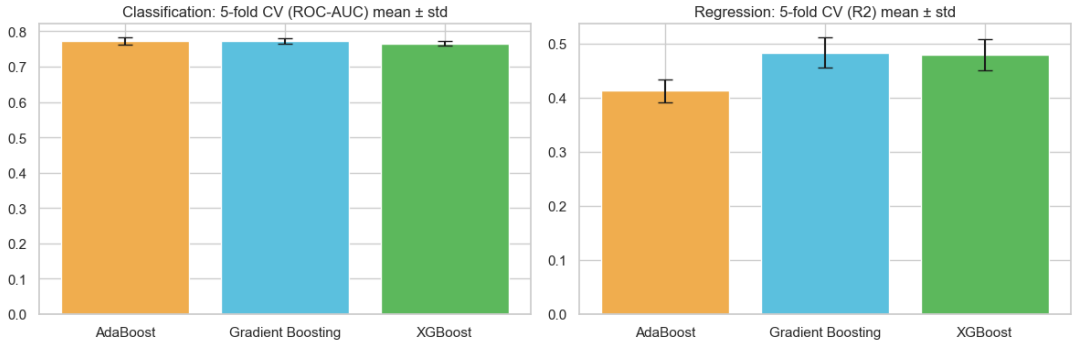


Figure 5: 5-fold CV results for AdaBoost, Gradient Boosting and XGBoost

Boosting model	Task	CV metric	Mean	Std
AdaBoost	Classification	ROC-AUC	0.7724	0.0093
Gradient Boosting	Classification	ROC-AUC	0.7721	0.0082
XGBoost	Classification	ROC-AUC	0.7653	0.0063
AdaBoost	Regression	R^2	0.4121	0.0212
Gradient Boosting	Regression	R^2	0.4829	0.0274
XGBoost	Regression	R^2	0.4787	0.0287

Interpretation: For regression, Gradient Boosting and XGBoost clearly beat AdaBoost (R^2 about 0.48 vs 0.41) because they use deeper trees and regularization. For classification the three are close; XGBoost is the most stable across folds (lowest std, 0.0063).

7. Part E: Voting & Stacking Ensembles (Tasks 18–21)

7.1 Tasks 18 and 19: Voting Classifier, Hard vs Soft

Base classifiers: Logistic Regression, KNN (k=15), Decision Tree (depth 5) and Random Forest (200 trees).

```
VotingClassifier(base_estimators(), voting='hard')
VotingClassifier(base_estimators(), voting='soft')
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.7423	0.6826	0.5846	0.6298	0.7928
KNN	0.6990	0.6442	0.4410	0.5236	0.7310
Decision Tree (depth 5)	0.7029	0.6588	0.4308	0.5209	0.7547
Random Forest	0.7260	0.6721	0.5256	0.5899	0.7748
Voting Classifier (Hard)	**0.7279**	**0.7034**	**0.4744**	**0.5666**	**0.6772**
Voting Classifier (Soft)	**0.7288**	**0.6812**	**0.5205**	**0.5901**	**0.7831**

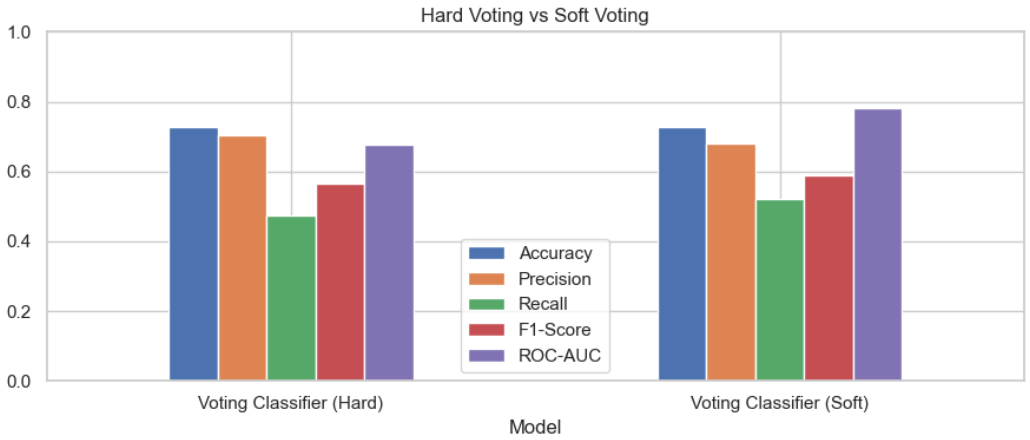


Figure 6: Hard Voting vs Soft Voting

Interpretation: Hard voting counts only class labels, so each model has an equal say and ROC-AUC can only be computed from labels (0.677). Soft voting averages probabilities, so confident models count more and a proper probability is produced, giving a much higher ROC-AUC (0.783).

7.2 Task 20: Stacking Classifier with a meta-learner

```
StackingClassifier(
    estimators=[('rf', RandomForest), ('xgb', XGBoost), ('knn', KNN), ('ada',
AdaBoost)],
    final_estimator=LogisticRegression(max_iter=1000),   # meta-learner
    cv=5, stack_method='predict_proba')
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Stacking Classifier	0.7346	0.6804	0.5513	0.6091	0.7871

7.3 Task 21: Stacking Regressor for score prediction

```
StackingRegressor(
    estimators=[('rf', RandomForest), ('xgb', XGBoost), ('knn', KNN), ('ada',
AdaBoost)],
    final_estimator=Ridge(alpha=1.0), cv=5)               # meta-learner
```

Model	MAE	RMSE	R^2
Stacking Regressor	8.1960	10.1850	0.4302

> **Interpretation:** Base models give out-of-fold predictions and the meta-learner (Logistic Regression or Ridge) learns how much to trust each of them. Stacking is the most flexible ensemble but also the slowest to train.

8. Part F: Model Evaluation & Comparison (Tasks 22, 29, 30)

8.1 Task 22: Classification model evaluation

Metrics: Accuracy, Precision, Recall, F1-Score and ROC-AUC on the 1,040-row test set. ROC-AUC (threshold independent) is the main ranking metric and F1 is the tie-breaker because the classes are imbalanced.

Model	Family	Acc	Prec	Recall	F1	ROC-AUC
Logistic Regression	Base	0.7423	0.6826	0.5846	0.6298	0.7928
Gradient Boosting Clf	**Boosting**	**0.7356**	**0.6706**	**0.5795**	**0.6217**	**0.7904**
Stacking Classifier	Stacking	0.7346	0.6804	0.5513	0.6091	0.7871
AdaBoost Classifier	Boosting	0.7298	0.6730	0.5436	0.6014	0.7870
Voting Classifier (Soft)	Voting	0.7288	0.6812	0.5205	0.5901	0.7831
XGBoost Classifier	Boosting	0.7317	0.6628	0.5795	0.6183	0.7812
Random Forest	Base	0.7260	0.6721	0.5256	0.5899	0.7748
Bagging Classifier	Bagging	0.7163	0.6401	0.5564	0.5953	0.7727
Decision Tree (depth 5)	Base	0.7029	0.6588	0.4308	0.5209	0.7547
KNN	Base	0.6990	0.6442	0.4410	0.5236	0.7310
Voting Classifier (Hard)	Voting	0.7279	0.7034	0.4744	0.5666	0.6772
Decision Stump	Base	0.6625	0.5572	0.4872	0.5198	0.6274
Decision Tree (single)	Base	0.6250	0.5000	0.5179	0.5088	0.6036

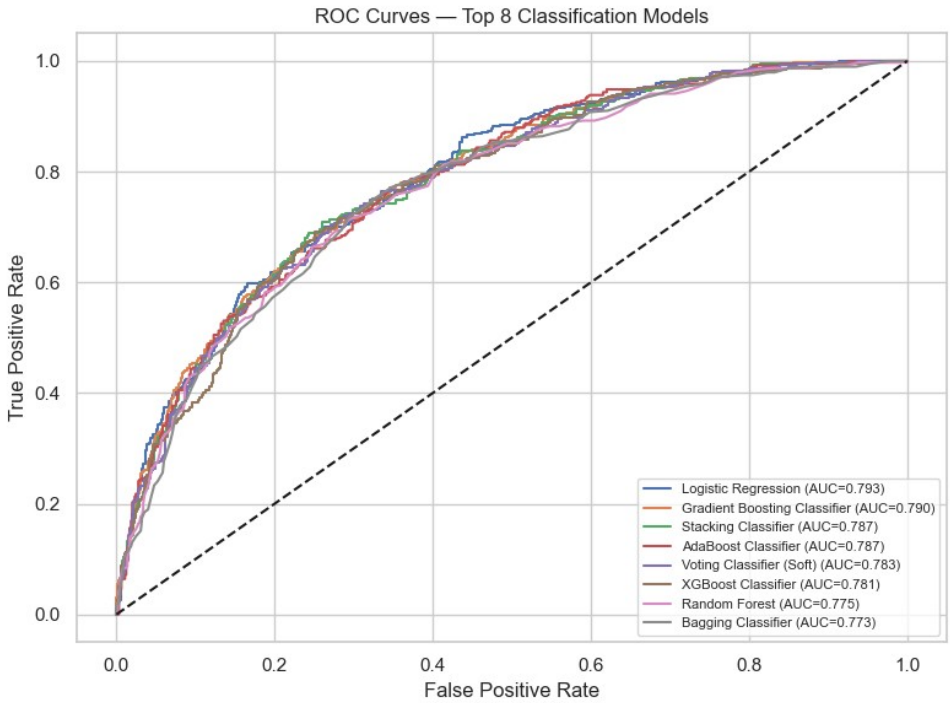


Figure 7: ROC curves of the top 8 classification models

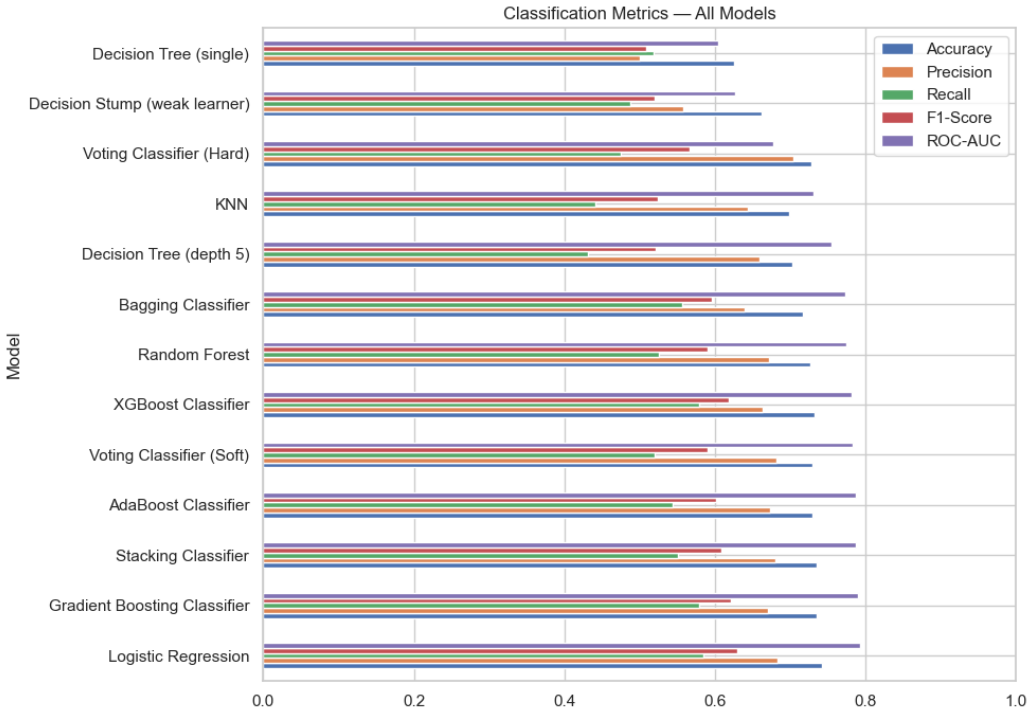


Figure 8: Classification metrics for all models

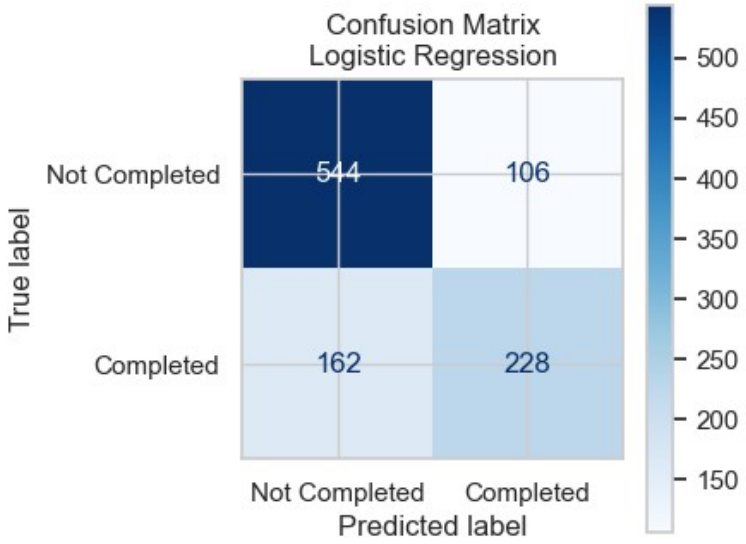


Figure 9: Confusion matrix of the top-ranked model overall (Logistic Regression)

Interpretation: The confusion matrix shows 544 true negatives, 228 true positives, 106 false positives and 162 false negatives, so the model finds about 58% of the students who actually complete the course. Among the ensemble models, Gradient Boosting has the best ROC-AUC and F1.

8.2 Task 29: Regression model evaluation

Metrics: MAE, RMSE and R^2. RMSE penalizes large errors more than MAE; R^2 shows how much of the variance in final_score the model explains.

Model	Family	MAE	RMSE	R^2
Gradient Boosting Regressor	**Boosting**	**8.1631**	**10.1468**	**0.4344**
Stacking Regressor	Stacking	8.1960	10.1850	0.4302
XGBoost Regressor	Boosting	8.2417	10.2149	0.4268
Bagging Regressor	Bagging	8.2389	10.2893	0.4184
AdaBoost Regressor	Boosting	8.6941	10.6530	0.3766
Shallow Tree depth=3	Base	9.3800	11.5976	0.2611
Decision Tree (single)	Base	11.1198	14.1222	-0.0955

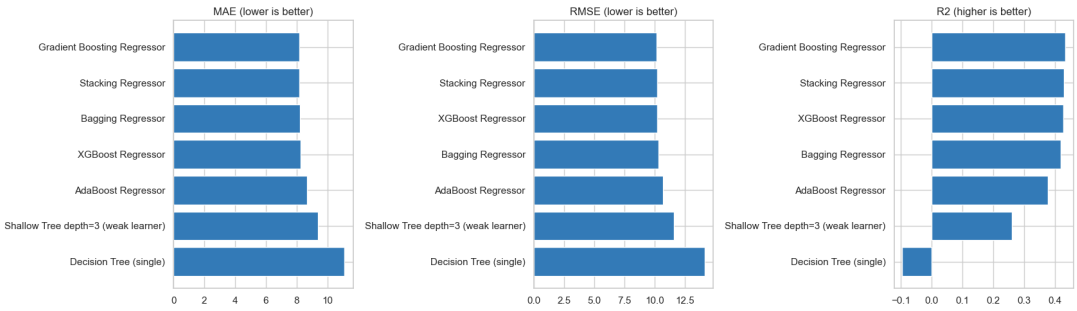


Figure 10: MAE, RMSE and R^2 for all regression models

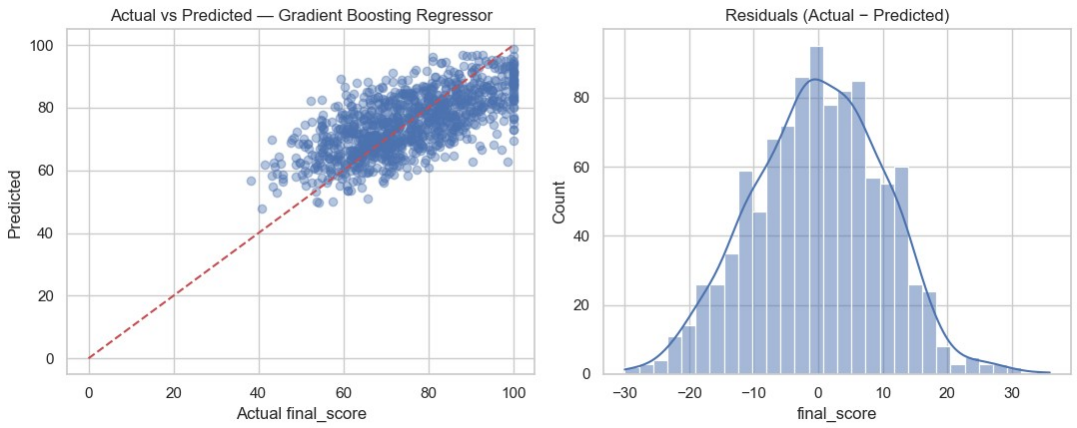


Figure 11: Actual vs predicted final_score and residuals (Gradient Boosting Regressor)

Interpretation: The points follow the red diagonal and the residuals are centred on zero, so the model is unbiased. The average error is about 8 marks out of 100 (MAE 8.16).

8.3 Task 30: Comparison of all ensemble techniques

Family	Best Accuracy	Best F1	Best ROC-AUC
Bagging	0.7163	0.5953	0.7727
Boosting	**0.7356**	**0.6217**	**0.7904**
Stacking	0.7346	0.6091	0.7871
Voting	0.7288	0.5901	0.7831

Family	Best RMSE	Best MAE	Best R^2
Bagging	10.2893	8.2389	0.4184
Boosting	**10.1468**	**8.1631**	**0.4344**
Stacking	10.1850	8.1960	0.4302

Best performing ensemble model for each task:

- **Classification:** Gradient Boosting Classifier (Accuracy 0.736, F1 0.622, ROC-AUC 0.790)
- **Regression:** Gradient Boosting Regressor (MAE 8.16, RMSE 10.15, R^2 0.434)

9. Part G: Final Analysis & Reporting (Tasks 31–32)

9.1 Impact of bagging vs boosting

	Single Tree	Bagging	AdaBoost	XGBoost
Classification ROC-AUC	0.604	0.773	0.787	0.781
Regression RMSE	14.12	10.29	10.65	10.21

Bagging reduces the variance of a single overfit tree, while boosting reduces bias by correcting errors step by step. Both give a large gain over a single tree; boosting is slightly better on the classification task.

9.2 Comparison of tree-based boosting algorithms

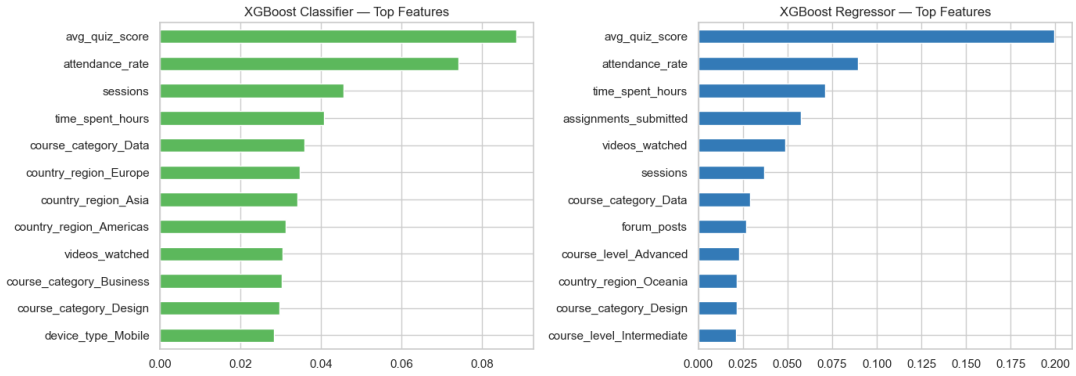


Figure 12: Top features of the XGBoost classifier and regressor

- **AdaBoost:** simple and fast with stumps, but weaker on regression and sensitive to noise.
- **Gradient Boosting:** best test results on both tasks in this project.
- **XGBoost:** regularized and the most stable across folds in classification; very close to Gradient Boosting on regression.

9.3 Advantages of voting and stacking

- **Voting** (Hard ROC-AUC 0.677, Soft 0.783): simple, uses diverse models; soft voting uses probabilities and is clearly better than hard voting.
- **Stacking** (ROC-AUC 0.787, RMSE 10.18): a meta-learner learns the best combination of base models; most flexible but the costliest to train.

> **Honest observation:** The plain Logistic Regression baseline (ROC-AUC 0.793) is very close to the best ensemble. This means the completion pattern is largely linear. The ensembles clearly beat the single tree (a large gain in stability), and for regression every ensemble beats the single and weak trees by a large margin.

9.4 Final model recommendation for deployment

Task	Recommended model	Key metrics
Classification (course completion)	Gradient Boosting Classifier	Accuracy 0.736, F1 0.622, ROC-AUC 0.790
Regression (final score)	Gradient Boosting Regressor	MAE 8.16, RMSE 10.15, R^2 0.434

Reasons: best test performance among ensembles, low variation across CV folds, fast prediction and a small saved pipeline that is easy to serve.

9.5 Business interpretation

- Students predicted as 'Not Completed' can be contacted early with reminders, mentoring or easier content to reduce drop-outs.
- The predicted final score helps the platform identify students who need extra support before the course ends.
- Ensemble models are more stable than a single model, which solves the manager's problem of unstable single-model results.
- The trained pipelines (preprocessing + model) are saved and served through a Streamlit app for real-time use.

10. Deployment

Streamlit application

The notebook saves the best pipelines (models/best_classifier.joblib and models / best_regressor.joblib) and a metadata file. The Streamlit app (app.py) loads them, takes the 16 student features as input, and shows the course completion probability and the predicted final score. A second tab shows the model comparison tables.

```
pip install -r requirements.txt
streamlit run app.py
```